UNIVERSIDADE FEDERAL DE MATO GROSSO
CAMPUS UNIVERSITÁRIO DO ARAGUAIA
CURSO DE CIÊNCIA DA COMPUTAÇÃO

RAISSA CAVALCANTI

PROGRAMA EM ASSEMBLY RISC-V PARA LEITURA, ORDENAÇÃO E PESQUISA DE VETORES DE NÚMEROS INTEIROS

BARRA DO GARÇAS - MT
2026

RAISSA CAVALCANTI

PROGRAMA EM ASSEMBLY RISC-V PARA LEITURA, ORDENAÇÃO E PESQUISA DE VETORES DE NÚMEROS INTEIROS

Trabalho apresentado à disciplina de Organização de Computadores do curso de Ciência da Computação da Universidade Federal de Mato Grosso, Campus Universitário do Araguaia, como requisito parcial de avaliação.

Docente: Prof. Linder Cândido.

BARRA DO GARÇAS - MT
2026

SUMÁRIO

1 INTRODUÇÃO.....4

2 OBJETIVO.....4

3 FUNDAMENTAÇÃO TEÓRICA.....5

3.1 A arquitetura RISC-V e a convenção de chamada.....5

3.2 Pilha, prólogo e epílogo de funções.....5

4 DESENVOLVIMENTO.....6

4.1 Organização dos dados na memória.....6

4.2 Leitura e validação da quantidade de elementos.....6

4.3 Leitura do vetor.....6

4.4 Impressão do vetor.....7

4.5 Soma, menor, maior valor e média.....7

4.6 Ordenação por bolha (bubble sort).....8

4.7 Pesquisa por busca binária.....8

5 TESTES E RESULTADOS.....9

6 CONSIDERAÇÕES FINAIS.....10

REFERÊNCIAS.....10

1 INTRODUÇÃO

A disciplina de Organização de Computadores aborda, entre outros temas, a relação entre a linguagem de alto nível e a linguagem de máquina executada pelo processador, bem como os mecanismos que permitem a um programa manipular dados, controlar o fluxo de execução e se comunicar com o sistema operacional. A arquitetura RISC-V, adotada como referência na disciplina, constitui um conjunto de instruções aberto e de projeto regular, amplamente utilizado tanto em contextos acadêmicos quanto industriais.

O presente trabalho consiste no desenvolvimento de um programa em linguagem Assembly RISC-V, executado em ambiente Linux de 64 bits, responsável por ler um conjunto de números inteiros fornecidos pelo usuário, calcular estatísticas simples sobre esse conjunto, ordená-lo por meio do algoritmo bolha (bubble sort) e localizar um valor específico utilizando busca binária. A implementação exercita diretamente os conceitos de acesso à memória, estruturas de repetição e decisão, definição e chamada de funções, uso da pilha de execução e integração controlada com a biblioteca padrão da linguagem C (libc), restrita às funções printf e scanf.

Este relatório está organizado da seguinte forma: a seção 2 apresenta o objetivo do trabalho; a seção 3 revisa, de forma sucinta, os fundamentos teóricos empregados; a seção 4 descreve o desenvolvimento de cada função do programa; a seção 5 apresenta os testes realizados e os resultados obtidos; e a seção 6 traz as considerações finais.

2 OBJETIVO

O objetivo deste trabalho é desenvolver, em linguagem Assembly RISC-V, um programa capaz de ler, processar, ordenar e pesquisar um conjunto de números inteiros, exercitando os seguintes conceitos:

a) acesso à memória e manipulação de vetores;

b) estruturas de repetição e de decisão;

c) implementação e chamada de funções;

d) convenção de chamada e retorno RISC-V/Linux;

e) utilização da pilha de execução;

f) integração de código Assembly com a biblioteca padrão C, restrita às funções de entrada e saída printf e scanf.

3 FUNDAMENTAÇÃO TEÓRICA

3.1 A arquitetura RISC-V e a convenção de chamada

RISC-V é uma arquitetura de conjunto de instruções (ISA) de projeto aberto, pertencente à filosofia RISC (Reduced Instruction Set Computer), caracterizada por um conjunto reduzido e regular de instruções, em contraposição às arquiteturas CISC (Complex Instruction Set Computer). O programa descrito neste relatório foi desenvolvido para a variante de 64 bits (RV64), na qual os trinta e dois registradores de propósito geral (x0 a x31) possuem 64 bits de largura.

A convenção de chamada adotada pelo Linux em sistemas RISC-V define o papel de cada registrador durante a chamada e o retorno de funções. Os registradores a0 a a7 são utilizados para a passagem de argumentos e, no caso de a0 e a1, também para os valores de retorno. O registrador ra armazena o endereço de retorno da função chamadora, e o registrador sp aponta para o topo da pilha de execução. Os registradores identificados pelo prefixo t (temporários) podem ser livremente sobrescritos por qualquer função chamada, ao passo que os registradores identificados pelo prefixo s (salvos) devem ter seu valor preservado por toda função que os utilize.

3.2 Pilha, prólogo e epílogo de funções

A pilha de execução é uma região de memória utilizada para armazenar, de forma temporária, o endereço de retorno e os registradores que uma função precisa preservar durante seu processamento, sobretudo quando essa função realiza, por sua vez, chamadas a outras funções. Toda função que necessita preservar algum registrador, ou que chama outra função, deve reservar espaço na pilha no início de sua execução (prólogo), por meio do decremento do registrador sp, e restaurar esse espaço ao término (epílogo), por meio do incremento equivalente de sp, imediatamente antes da instrução de retorno.

Uma função que não realiza nenhuma chamada a outra função é denominada função-folha. Quando uma função-folha não faz uso de registradores salvos, ela dispensa a construção de prólogo e epílogo, uma vez que não há necessidade de preservar o endereço de retorno nem qualquer registrador salvo. Essa distinção foi observada na organização das funções descritas na seção 4.

4 DESENVOLVIMENTO

O programa foi estruturado em nove rotinas: a função principal (main) e oito funções auxiliares, cada uma responsável por uma etapa específica do processamento do vetor. Nesta seção, descreve-se a organização dos dados na memória e a lógica empregada em cada uma dessas rotinas.

4.1 Organização dos dados na memória

Os dados do programa foram alocados na seção .bss do arquivo executável, reservada para variáveis globais não inicializadas. Foram definidas três variáveis: a quantidade de elementos informada pelo usuário (n), o valor a ser pesquisado no vetor (valor_para_consulta) e o próprio vetor de números inteiros (vetor), dimensionado para comportar até cem elementos de quatro bytes cada, totalizando quatrocentos bytes.

4.2 Leitura e validação da quantidade de elementos

A função main inicia solicitando ao usuário a quantidade de elementos a serem processados, por meio da função printf, seguida da leitura do valor informado com a função scanf. Em seguida, o valor lido é validado em um laço de repetição: caso esteja fora do intervalo de um a cem, uma mensagem de erro é exibida e uma nova leitura é solicitada, repetindo-se esse procedimento até que um valor válido seja fornecido. Essa validação exercita diretamente uma estrutura de decisão combinada a uma estrutura de repetição, ambas implementadas por meio de instruções de desvio condicional.

4.3 Leitura do vetor

A leitura dos elementos do vetor é realizada pela função read_vector, que recebe como argumentos o endereço base do vetor (registrador a0) e a quantidade de elementos a ler (registrador a1). Como essa função realiza, em laço, chamadas repetidas às funções printf e scanf da biblioteca padrão, torna-se necessário preservar o endereço base, a quantidade de elementos e o índice corrente do laço em registradores salvos (s0, s1 e s2, respectivamente), uma vez que uma chamada de função pode sobrescrever livremente os registradores de argumento e os registradores temporários. O Código-fonte 1 apresenta o trecho correspondente.

Código-fonte 1 - trecho da função read_vector

```
    read_vector:
```

```
    addi sp, sp, -48
    sd   ra, 40(sp)
    sd   s0, 32(sp)      # s0 = endereco base do vetor
    sd   s1, 24(sp)      # s1 = quantidade de elementos
    sd   s2, 16(sp)      # s2 = indice do laco

    mv   s0, a0
    mv   s1, a1
    li   s2, 0

rv_loop:
    bge  s2, s1, rv_fim
    la   a0, fmt_prompt_elem
    addi a1, s2, 1
    call printf
    la   a0, fmt_scan_int
    slli t0, s2, 2
    add  a1, s0, t0      # a1 = endereco de vetor[i]
    call scanf
    addi s2, s2, 1
    j    rv_loop
```

Fonte: elaborado pela autora (2026).

A cada iteração, o endereço do i-ésimo elemento é calculado multiplicando-se o índice por quatro, por meio de um deslocamento aritmético de dois bits à esquerda (slli), e somando-se o resultado ao endereço base do vetor, uma vez que cada elemento ocupa quatro bytes na memória.

4.4 Impressão do vetor

A função print_vector segue a mesma lógica de preservação de registradores descrita na subseção anterior, substituindo a chamada a scanf por uma chamada a printf com o formato "%d ", exibindo cada elemento do vetor seguido de um espaço em branco. Ao término do laço, uma quebra de linha adicional é impressa para separar a listagem dos elementos das informações subsequentes.

4.5 Soma, menor, maior valor e média

As funções sum_vector, min_vector e max_vector não realizam nenhuma chamada a outras funções e, por essa razão, foram implementadas como funções-folha, dispensando a construção de prólogo e epílogo. A função sum_vector percorre o vetor acumulando a soma dos elementos em um registrador temporário; min_vector e max_vector percorrem o vetor a partir do segundo elemento, comparando cada candidato ao valor mínimo ou máximo corrente por meio de instruções de desvio condicional (bge e ble, respectivamente), atualizando o registrador correspondente sempre que um valor mais extremo é encontrado.

A função avg_vector, por sua vez, calcula a média inteira do vetor chamando internamente a função sum_vector e dividindo o resultado pela quantidade de elementos, por meio da instrução div. Como essa função realiza uma chamada a outra função, foi necessário reservar espaço na pilha para preservar o endereço de retorno e a quantidade de elementos, conforme ilustrado no Código-fonte 2.

Código-fonte 2 - trecho da função avg_vector

```
avg_vector:
    addi sp, sp, -16
    sd   ra, 8(sp)
    sd   s0, 0(sp)        # s0 = quantidade de elementos

    mv   s0, a1
    call sum_vector       # devolve a soma em a0
    div  a0, a0, s0       # media inteira = soma / n

    ld   ra, 8(sp)
    ld   s0, 0(sp)
    addi sp, sp, 16
    ret
```

Fonte: elaborado pela autora (2026).

A divisão inteira realizada pela instrução div trunca o resultado em direção a zero, comportamento equivalente à divisão de inteiros na linguagem C, utilizada como referência no enunciado do trabalho.

4.6 Ordenação por bolha (bubble sort)

A ordenação do vetor foi implementada na função sort_vector por meio do algoritmo bolha, conforme exigido pelo enunciado do trabalho. O algoritmo compara, em um laço aninhado, cada par de elementos adjacentes do vetor e os troca de posição sempre que estiverem fora de ordem, repetindo esse procedimento até que o vetor esteja completamente ordenado em ordem crescente. Por não realizar nenhuma chamada a outra função, sort_vector também foi implementada como função-folha, utilizando exclusivamente registradores temporários.

4.7 Pesquisa por busca binária

A pesquisa de um valor no vetor é realizada pela função find_value por meio do algoritmo de busca binária, que pressupõe um vetor previamente ordenado. Por esse motivo, a chamada a essa função, em main, ocorre necessariamente após a chamada a sort_vector. O algoritmo mantém dois índices, denominados limite inferior e limite superior, e a cada iteração calcula o índice central do intervalo corrente, comparando o elemento correspondente ao valor

procurado: caso sejam iguais, o índice é retornado; caso o elemento seja menor que o valor procurado, o limite inferior é ajustado para a posição seguinte ao índice central; caso contrário, o limite superior é ajustado para a posição anterior. O processo se repete até que o valor seja localizado ou até que o limite inferior ultrapasse o limite superior, situação em que a função retorna o valor -1, indicando que o elemento não foi encontrado.

5 TESTES E RESULTADOS

O programa foi compilado com o conjunto de ferramentas riscv64-linux-gnu-gcc e executado sob emulação de usuário por meio do utilitário qemu-riscv64-static, em ambiente Linux. Foram realizados testes cobrindo os principais cenários de uso do programa, incluindo casos de borda, de modo a verificar a correção de cada uma das funções implementadas e da integração entre elas. A Tabela 1 resume os casos de teste executados e os resultados observados.

Tabela 1 - Casos de teste executados e resultados observados

Nº	**Entrada**	**Resultado observado**
1	N = 5; vetor {10, 3, 8, 1, 9}; busca por 8.	Soma 31; menor 1; maior 10; média 6; vetor ordenado {1, 3, 8, 9, 10}; valor encontrado no índice 2.
2	N = 4; vetor {5, 2, 9, 1}; busca por 100.	Soma 17; menor 1; maior 9; média 4; vetor ordenado {1, 2, 5, 9}; valor não encontrado.
3	N = 0, depois N = 101, depois N = 3; vetor {7, 7, 7}; busca por 7.	A validação de N rejeitou os dois valores fora do intervalo antes de aceitar N = 3; soma 21; menor 7; maior 7; média 7; valor encontrado.
4	N = 1; vetor {42}; busca por 42.	Soma, menor, maior e média iguais a 42; valor encontrado no índice 0.
5	N = 5; vetor {-10, 3, -8, 0, 9}; busca por -10.	Soma -6; menor -10; maior 9; média -1 (divisão truncada em direção a zero); vetor ordenado {-10, -8, 0, 3, 9}; valor encontrado no índice 0.
6	N = 6; vetor já ordenado {1, 2, 3, 4, 5, 6}; busca por 6.	Vetor permanece ordenado; valor encontrado no índice 5.
7	N = 5; vetor em ordem decrescente {50, 40, 30, 20, 10}; busca por 10.	Vetor ordenado corretamente para {10, 20, 30, 40, 50}; valor encontrado no índice 0.
8	N = 100, valores aleatórios entre -500 e 500; busca por um valor presente no vetor.	Vetor ordenado corretamente ao longo de todas as cem posições; valor localizado corretamente.
9	N = 12, informado interativamente pelo teclado; vetor {2, 32, 100, 54, 101, 23, 12, 32, 42, 65, 75, 86}.	Soma 624; menor 2; maior 101; média 52; vetor ordenado {2, 12, 23, 32, 32, 42, 54, 65, 75, 86, 100, 101}, preservando corretamente os valores repetidos.

Fonte: elaborado pela autora (2026).

Além dos testes apresentados na Tabela 1, o programa foi submetido a uma rotina automatizada de verificação, na qual duzentas execuções foram realizadas com valores de entrada gerados aleatoriamente, variando a quantidade de elementos entre um e cem, incluindo valores negativos e positivos, e comparando a soma, o menor valor, o maior valor, a média inteira, o vetor ordenado e o resultado da pesquisa binária com os valores calculados de forma independente. Nenhuma divergência foi observada nessas duzentas execuções, o que fornece evidência adicional de que a implementação atende corretamente aos requisitos especificados.

Por fim, verificou-se que a validação do intervalo de N funciona corretamente mesmo diante de múltiplas entradas inválidas consecutivas, repetindo a solicitação até que um valor dentro do intervalo de um a cem seja informado, conforme demonstrado no caso de teste número três.

6 CONSIDERAÇÕES FINAIS

O desenvolvimento deste trabalho permitiu exercitar, de forma prática, os principais conceitos relacionados à programação em linguagem Assembly sobre a arquitetura RISC-V, em especial a convenção de chamada de funções, o uso da pilha de execução para a preservação de registradores entre chamadas, a distinção entre funções-folha e funções que realizam chamadas aninhadas, e a integração controlada entre código Assembly e a biblioteca padrão da linguagem C.

Os testes realizados, descritos na seção 5, confirmam que o programa atende integralmente aos requisitos especificados no enunciado do trabalho, apresentando corretamente o vetor original, a soma, o menor e o maior valor, a média inteira e o vetor ordenado, além de localizar corretamente, por meio de busca binária, um valor pesquisado pelo usuário, informando sua ausência quando aplicável.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

CÂNDIDO, Linder. Organização de Computadores: notas de aula. Barra do Garças: Universidade Federal de Mato Grosso, Campus Universitário do Araguaia, 2026.

RISC-V INTERNATIONAL. The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA. [S.l.]: RISC-V International, 2024.

FREE SOFTWARE FOUNDATION. Using as: The GNU Assembler. Boston: Free Software Foundation, 2023.